

# S01 - Evidencia individual de pedidos-ms

Documento preparado para exportación PDF. Las capturas deben agregarse con reloj del sistema y usuario o perfil visible.

**Nombre:** Frank Alexander

**Equipo:** En proceso

**Sesión:** S01 - Construcción de un servicio base para un sistema distribuido

**Rol o aporte:** Implementación individual de `pedidos-ms`, CRUD, persistencia y pruebas.

**GitHub:** <https://github.com/puma13H/pedidos-ms-en-proceso>

## Evidencia técnica

### 1. Microservicio delimitado

Caso demostrado: una tienda registra el pedido de Juan Perez por una laptop Lenovo y un mouse, por un total de S/ 2850.00. `pedidos-ms` gestiona el cliente del pedido, la descripción, el total, el estado y la fecha de creación. `pagatu-catalogo-ms` mantiene los productos y categorías; separar ambos dominios evita que `pedidos-ms` lea las tablas del catálogo.

Estados usados: `PENDIENTE`, `CONFIRMADA`, `PREPARANDO`, `ENVIADA`, `ENTREGADA` y `CANCELADA`.

**Captura 1:** insertar captura del árbol del proyecto y del código de `Orden`.

### 2. PostgreSQL y Flyway

La migración `V1__create_orden_tables.sql` crea la tabla `ordenes` con cliente, descripción, total, estado y fecha de creación. Hibernate usa `ddl-auto: validate`; Flyway define el esquema y Hibernate solo lo comprueba.

**Captura 2:** insertar `docker ps` mostrando PostgreSQL en `15433`.

**Captura 3:** insertar log con `Successfully applied 1 migration` y consulta `psql`.

```
docker exec -it pagatu-postgres-pedidos-dev psql -U pagatu -d pagatu_orden_db -c "SELECT id,
```

```
cliente_nombre, descripcion, total, estado, fecha_creacion FROM ordenes;"
```

### 3. REST y Swagger

El CRUD se prueba con `/api/v1/ordenes`.

```
$body = '{"clienteNombre":"Juan Perez","descripcion":"Laptop Lenovo +
Mouse","total":2850.00,"estado":"PENDIENTE"}'
$orden = Invoke-RestMethod -Method Post -Uri http://localhost:8080/api/v1/ordenes -
ContentType "application/json" -Body $body
$orden
Invoke-RestMethod http://localhost:8080/api/v1/ordenes
Invoke-RestMethod http://localhost:8080/api/v1/ordenes/$($orden.id)
$update = '{"clienteNombre":"Juan Perez","descripcion":"Laptop Lenovo +
Mouse","total":2850.00,"estado":"CONFIRMADA"}'
Invoke-RestMethod -Method Put -Uri http://localhost:8080/api/v1/ordenes/$($orden.id) -
ContentType "application/json" -Body $update
Invoke-RestMethod -Method Delete -Uri http://localhost:8080/api/v1/ordenes/$($orden.id)
```

La evidencia debe mostrar la respuesta `201 Created`, el cambio a `CONFIRMADA` y el registro actualizado en PostgreSQL antes del DELETE.

**Captura 4:** insertar CRUD por PowerShell.

**Captura 5:** insertar Swagger en `http://localhost:8080/swagger-ui/index.html`.

### 4. Health, métricas y escalamiento

```
.\mvnw.cmd spring-boot:run
.\mvnw.cmd spring-boot:run "-Dspring-boot.run.arguments=--server.port=8081"

Invoke-RestMethod http://localhost:8080/actuator/health
Invoke-RestMethod http://localhost:8081/actuator/health
Invoke-RestMethod http://localhost:8080/actuator/metrics
Invoke-RestMethod http://localhost:8081/actuator/metrics
```

La instancia única es stateless porque el estado del pedido se guarda en PostgreSQL y no en la memoria de la aplicación. Resultado verificado: health `UP`, métricas HTTP 200 y Swagger HTTP 200 en 8080.

**Captura 6:** insertar Docker Desktop mostrando una instancia `pedidos-ms` en 8080 y PostgreSQL, junto con health `UP`.

### 5. Documentación reproducible

La ejecución está documentada en [README.md del repositorio](#), con comandos DEV, PostgreSQL, CRUD, Docker y diagnóstico del hallazgo técnico.

## Error o hallazgo

Durante la primera prueba PROD, las instancias conectaban a PostgreSQL pero fallaban con `Schema validation: missing table [ordenes]`. El diagnóstico mostró que no aparecía ningún evento de Flyway y `psql` devolvía `Did not find any relations`. Se agregó `spring-boot-starter-flyway`, se recreó el volumen y Flyway aplicó V1 correctamente antes de que Hibernate validara el esquema.

## Reflexión técnica

Un microservicio reproducible reduce las diferencias entre equipos y ambientes. DEV permite cambios rápidos con Maven Wrapper y PostgreSQL aislado. PROD local se acerca al empaquetado real mediante Docker. Ser stateless permite ejecutar copias sin perder información local. Cada instancia puede usar un puerto distinto. En un entorno real, un Gateway o balanceador repartiría las solicitudes. Flyway controla la evolución de la base propia del servicio.

## Anexo: Feedback de la sesión

**Aprendizaje más importante:** [completar]

**Punto más confuso:** [completar]

**Pregunta para la siguiente clase:** [completar]

**Nivel:** [Entendido / Más o menos / Necesito ayuda]

**Cómo mejorar mi comprensión:** [completar]

**Compromiso:** [Muy Comprometido/a / Comprometido/a / Poco Comprometido/a]

**Satisfacción (1-10):** [completar]